

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of a Convolutional Neural Network (CNN) for Image Classification

Name	Vinay Sathish
Roll Number	24011101126
Degree & Branch	B.Tech Artificial Intelligence & Data Science (Section-B)
Subject Code & Name	CS3807 – Deep Learning Laboratory

1. Objective

Implement a Convolutional Neural Network (CNN) using TensorFlow/Keras for multi-class image classification on the CIFAR-10 dataset. Learn convolution and pooling operations, CNN architecture design, model training and evaluation, and visualization of learned convolutional feature maps.

2. Learning Outcomes

Should be able to:

1. Explain the architecture and working of a Convolutional Neural Network.
2. Preprocess RGB image datasets for CNN input.
3. Build, compile, and train a CNN using Keras.
4. Evaluate classification performance using standard metrics.
5. Visualize and interpret learned convolutional feature maps.

3. Background Theory

A Convolutional Neural Network (CNN) is a deep learning architecture designed for grid-structured data such as images. Unlike a fully connected MLP, a CNN uses learnable convolutional filters that slide over the input to extract local spatial features (edges, textures, shapes), followed by pooling layers that reduce spatial dimensionality while retaining the most salient features.

A CNN is generally composed of the following layer types:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

Convolution operation:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

where I is the input feature map and K is the convolutional kernel.

Output feature map size:

$$\frac{N - F + 2P}{S} + 1$$

N Input Size

F Filter Size

P Padding

S Stride

Max Pooling reduces each spatial region to its maximum activation, providing translation invariance and reducing the number of parameters in subsequent layers.

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss (Sparse Categorical Cross-Entropy):

$$\mathcal{L} = -\log(\hat{y}_{y_{\text{true}}})$$

Architecture used in this experiment:

Conv2D(32, 3×3 , ReLU) $\rightarrow$ MaxPool(2×2) $\rightarrow$ Conv2D(64, 3×3 , ReLU) $\rightarrow$ MaxPool(2×2) $\rightarrow$

Conv2D(128, 3×3 , ReLU) $\rightarrow$ Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10, Softmax)

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

3.2 Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

3.3 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

3.4 Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448. (Note: this toy example is separate from the actual 3-layer, 1,143,242-parameter architecture trained in this experiment; see Section 6, Task 2, for the real parameter breakdown.)

4. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10 (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)
- Image Size: $32 \times 32 \times 3$ (RGB)

Result: The dataset was loaded via `tf.keras.datasets.cifar10`. The loaded shapes matched specification exactly: training set (50000, 32, 32, 3) and testing set (10000, 32, 32, 3).

5. Data Preprocessing

1. Load CIFAR-10.
2. Display sample images with class labels.
3. Plot class distribution.
4. Normalize pixel values to $[0, 1]$.

Result: Pixel values were cast to `float32` and divided by 255, mapping the raw $[0, 255]$ integer range to $[0, 1]$. No flattening was required, since Conv2D layers operate directly on the 3-dimensional (32, 32, 3) image tensors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load CIFAR-10.
- Print dataset dimensions.
- Display ten sample images with labels.
- Plot class distribution.

Result: Training Images: (50000, 32, 32, 3); Testing Images: (10000, 32, 32, 3). Sample images and the class distribution plot are provided in Section 8.

Task 2: Model Construction

Construct a CNN with

- Conv2D(32 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(64 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(128 filters, 3×3 , ReLU, same padding)
- Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10, Softmax)

Result:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 128)	1,048,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

- Total params: 1,143,242 (4.36 MB)
- Trainable params: 1,143,242 (4.36 MB)
- Non-trainable params: 0 (0.00 B)

Task 3: Model Training

Compile using

- Optimizer: Adam
- Loss: Sparse Categorical Cross-Entropy
- Metric: Accuracy

Train for 10 epochs with batch size 64, using a 20% validation split.

Result: Training accuracy rose from 37.53% (epoch 1) to 78.01% (epoch 10), while validation accuracy rose from 51.40% to 73.58% over the same period. The epoch-wise training log is summarized below.

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss
1	0.3753	1.6973	0.5140	1.3407
2	0.5245	1.3247	0.6113	1.0913
3	0.5909	1.1522	0.6466	0.9904
4	0.6377	1.0334	0.6854	0.8973
5	0.6702	0.9386	0.6975	0.8535
6	0.6962	0.8681	0.7189	0.8083
7	0.7212	0.7896	0.7142	0.8358
8	0.7488	0.7169	0.7260	0.8158
9	0.7646	0.6727	0.7265	0.8183
10	0.7801	0.6148	0.7358	0.7831

Full accuracy/loss curves are provided in Section 8.

Task 4: Model Evaluation

Compute

- Test Accuracy and Loss
- Precision, Recall, F1-score (per class and averaged)
- Confusion Matrix

- Classification Report

Result: On the held-out test set (10,000 images):

Metric	Value
Test Accuracy	0.7345
Test Loss	0.8070
Precision (macro)	0.74
Recall (macro)	0.73
F1-score (macro)	0.74

Classification Report:

Class	Precision	Recall	F1-score	Support
Airplane	0.79	0.77	0.78	1000
Automobile	0.88	0.83	0.86	1000
Bird	0.67	0.59	0.63	1000
Cat	0.52	0.58	0.55	1000
Deer	0.70	0.68	0.69	1000
Dog	0.59	0.66	0.62	1000
Frog	0.79	0.79	0.79	1000
Horse	0.77	0.80	0.78	1000
Ship	0.84	0.86	0.85	1000
Truck	0.83	0.79	0.81	1000
Accuracy	0.73			10000
Macro avg	0.74	0.73	0.74	10000
Weighted avg	0.74	0.73	0.74	10000

The Automobile and Ship classes are the strongest performers (F1-score 0.86 and 0.85), while Cat is by far the weakest (F1-score 0.55), consistently confused with visually similar animal classes such as Dog and Bird. The confusion matrix is provided in Section 8.

Task 5: Model Interpretation

- Compute total trainable parameters.
- Visualize convolutional feature maps for a sample test image.
- Display a sample prediction (actual vs. predicted label).

Result: The model has 1,143,242 trainable parameters. Feature maps were extracted from all three convolutional layers for a single test image, and a qualitative prediction example is shown in Section 8.

7. Plots Generated

The following plots were generated during the experiment.

Plot	Purpose	Expected Inference
Sample Images	Dataset Familiarization	Visually confirm image content, resolution, and label correctness.
Class Distribution	Balance Check	Verify all classes are equally represented.
Training vs Validation Accuracy & Loss	Learning Behaviour	Assess convergence and detect overfitting.
Confusion Matrix	Performance Evaluation	Identify which classes are most frequently confused.
Convolutional Feature Maps (per layer)	Model Interpretability	Visualize what spatial features each convolutional layer has learned.
Sample Prediction	Qualitative Evaluation	Spot-check a single prediction against the ground truth label.

7.1 Generated Plots and Inference

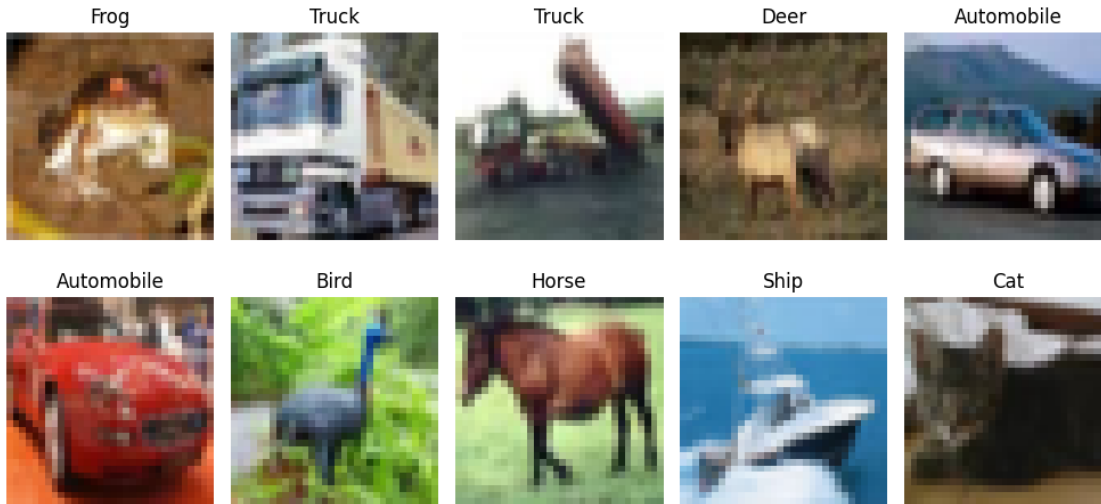


Figure 1: Sample CIFAR-10 images

Inference: The ten sample images confirm the dataset consists of low-resolution (32×32) RGB natural images spanning both vehicles (automobile, truck, ship, airplane) and animals (bird, cat, deer, dog, frog, horse), with far more visual complexity and background clutter than a grayscale dataset like Fashion-MNIST.

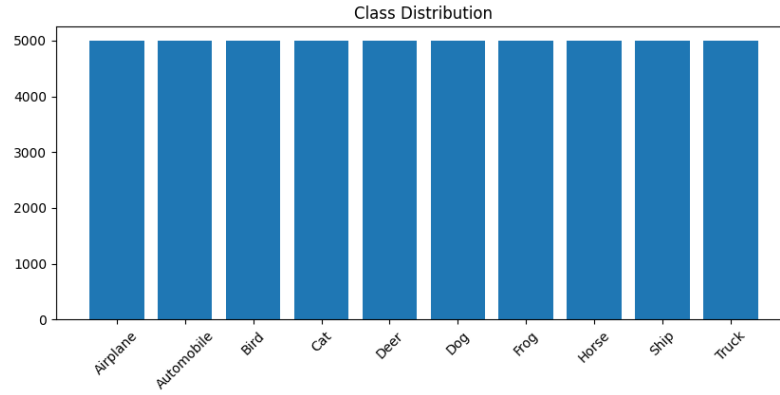


Figure 2: Class distribution (training set)

Inference: All 10 classes are equally represented in the training set (5,000 images each), confirming the dataset is perfectly balanced and that accuracy is a fair evaluation metric without needing class reweighting.

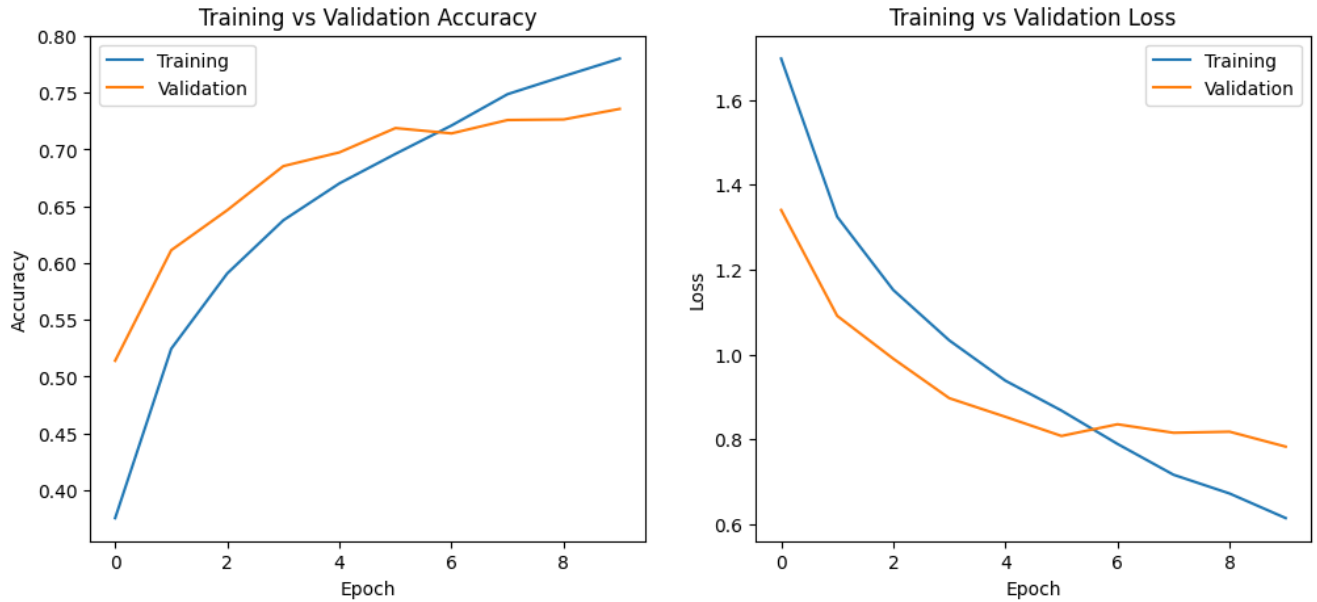


Figure 3: Training vs. validation accuracy and loss

Inference: Training accuracy climbs steadily and consistently through all 10 epochs, while validation accuracy improves quickly at first but grows more slowly after epoch 6 and briefly dips at epoch 7, and validation loss flattens out and even ticks upward around epochs 7 to 9 while training loss keeps falling, which could be a sign that the model is starting to overfit and would likely benefit from more epochs, regularization (e.g., data augmentation), or early stopping.

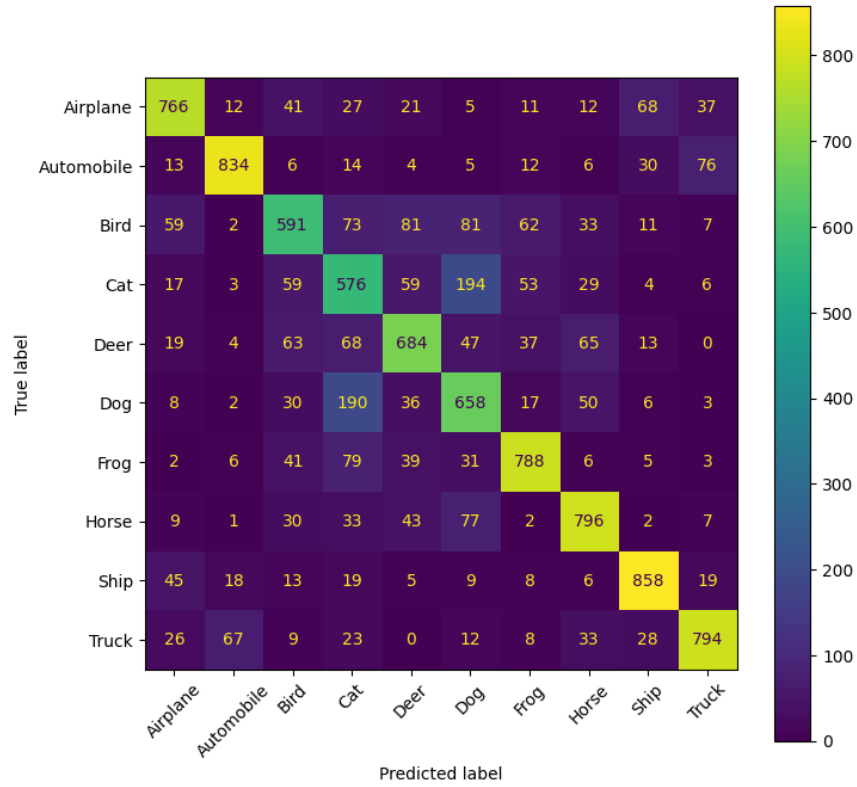


Figure 4: Confusion matrix (test set)

Inference: The strongest diagonal entries belong to Automobile, Ship, and Truck, which have visually distinct shapes and backgrounds, while Cat, Dog, and Bird show substantial mutual confusion, consistent with these being small, texture-similar animal classes that are hard to distinguish at 32×32 resolution.

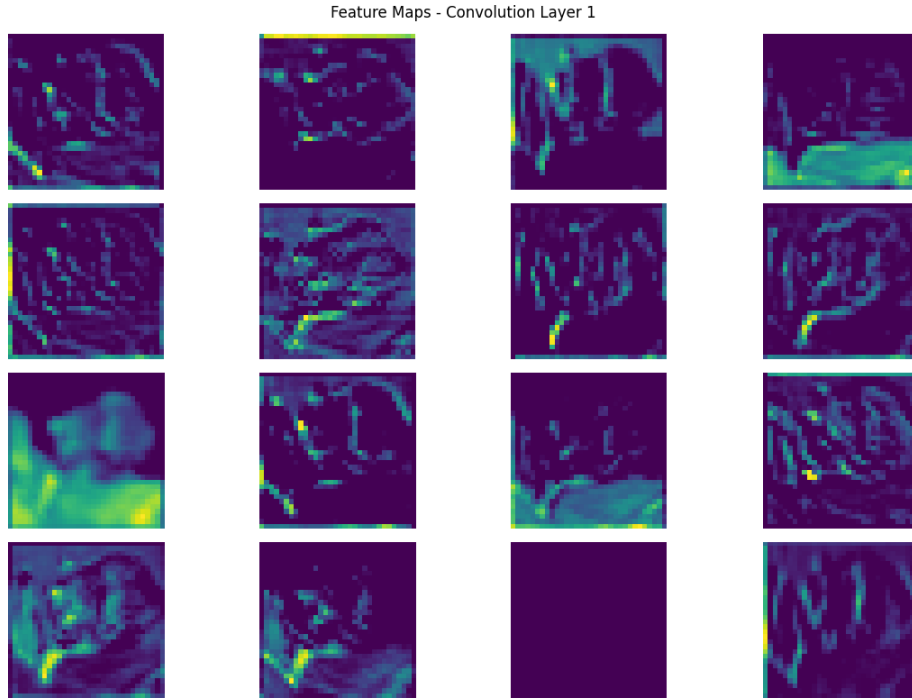


Figure 5: Feature maps – Convolution Layer 1

Inference: The first convolutional layer's 32 filters extract low-level, generic features such as edges, colour contrasts, and coarse object outlines, with most filters still preserving a recognizable silhouette of the input image.

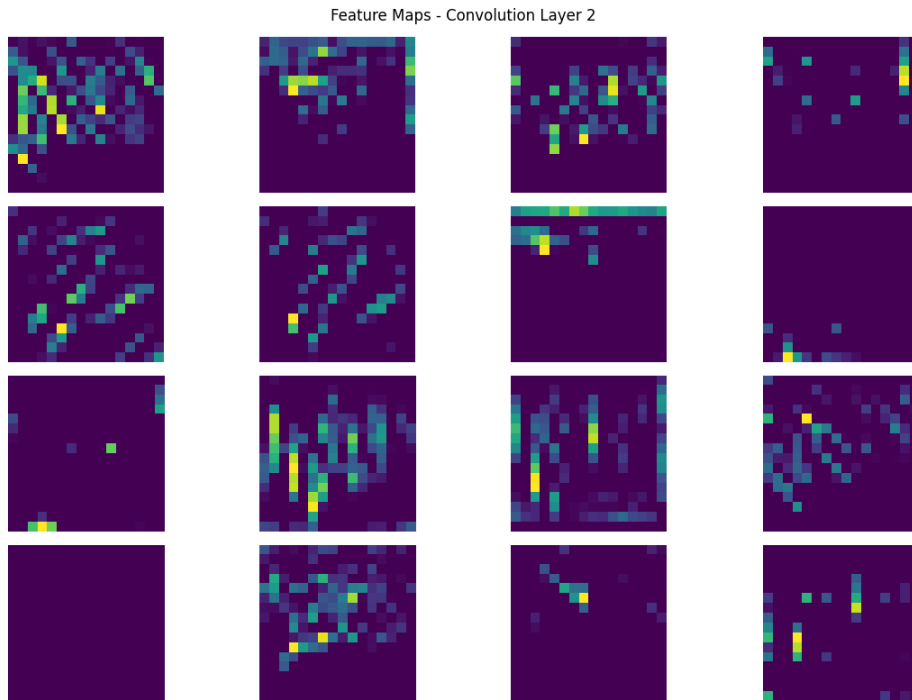


Figure 6: Feature maps – Convolution Layer 2

Inference: By the second convolutional layer, the feature maps are more abstract and spatially

coarser (reflecting the intermediate max-pooling step), highlighting specific textured regions rather than the whole object outline.

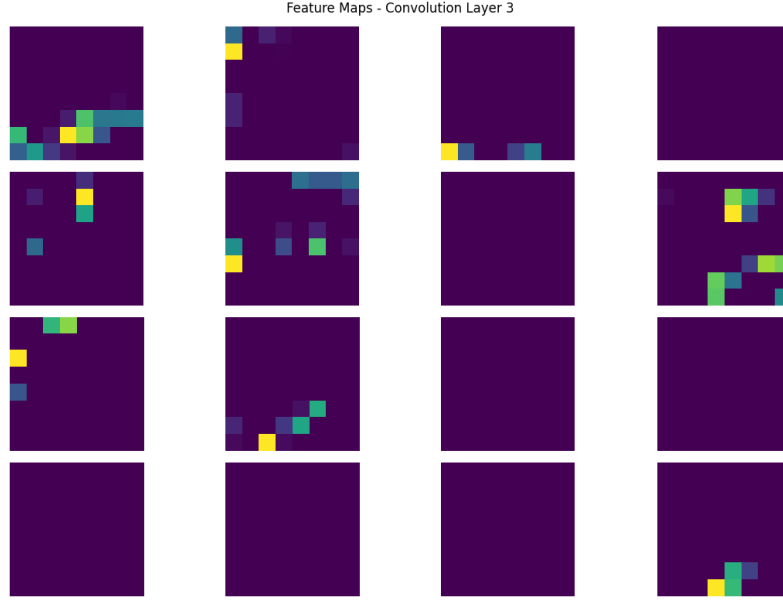


Figure 7: Feature maps – Convolution Layer 3

Inference: The third convolutional layer’s feature maps are the most abstract and spatially compressed of the three, with several filters appearing to activate on highly specific, localized patterns, consistent with the general CNN principle that deeper layers encode increasingly high-level, task-specific features.



Figure 8: Sample prediction (test image)

Inference: For this sample test image the model correctly predicted “Cat”, matching the ground truth label, despite Cat being the class with the lowest overall F1-score (0.55), which illustrates that the model succeeds on some individual cat images while still struggling with the class as a whole.

8. Results Summary

Dataset Size	60,000 (50,000 train / 10,000 test)
Image Size	$32 \times 32 \times 3$
Architecture	Conv(32) $\rightarrow$ Pool $\rightarrow$ Conv(64) $\rightarrow$ Pool $\rightarrow$ Conv(128) $\rightarrow$ Dense(128) $\rightarrow$ Dropout(0.5)
Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Batch Size	64
Epochs	10
Trainable Parameters	1,143,242
Final Training Accuracy	0.7801
Final Validation Accuracy	0.7358
Test Accuracy	0.7345
Test Loss	0.8070
Macro F1-score	0.74
Best-performing Class	Automobile (F1 = 0.86)
Worst-performing Class	Cat (F1 = 0.55)

9. Source Code

The complete source code used for this experiment (dataset loading, preprocessing, CNN construction and training, evaluation, and feature map visualization) is listed below.

<https://github.com/vinukohshawenn/deeplearningexperiments/tree/main/convolutional-neural-network>

10. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution uses small, shared kernels that scan the entire image, so the number of parameters depends on the kernel size rather than the image size, and the same feature detector is reused at every spatial location (parameter sharing and translation invariance). A fully connected layer would instead need a unique weight for every pixel-to-neuron connection, which is computationally infeasible for images and ignores their spatial structure – as illustrated by the CNN in this experiment (1,143,242 parameters) versus how many an equivalent MLP directly on $32 \times 32 \times 3$ pixels would require.

2. How does stride affect the feature map size?

A larger stride moves the kernel by more pixels between applications, which reduces the size of the output feature map (per $\frac{N-F+2P}{S} + 1$) and reduces computation, at the cost of a coarser, lower-resolution representation.

3. What is the role of padding?

Padding adds extra pixels (usually zeros) around the input border. “Same” padding preserves the spatial dimensions of the input in the output feature map (as used for all Conv2D layers in this experiment), while “valid” padding (no padding) shrinks the output and can lose information at the borders after repeated convolutions.

4. Why is pooling used?

Pooling (Max Pooling here) reduces the spatial dimensions of feature maps, which lowers the number of parameters and computation in subsequent layers, provides a degree of translation invariance, and helps control overfitting.

5. How do feature maps represent image characteristics?

Each feature map is the response of one learned filter across the image: early-layer feature maps (Figure 5) capture low-level features like edges and colour contrasts and still resemble the input’s silhouette, while deeper-layer feature maps (Figures 6 and 7) become more abstract, spatially compressed, and selective for specific, high-level patterns relevant to classification.

6. Why do CNNs require fewer parameters than MLPs?

Because of parameter sharing (the same small kernel is reused across all spatial positions) and local connectivity (each neuron only connects to a small receptive field instead of the entire input), CNNs need far fewer weights than a fully connected MLP operating directly on raw pixels, which is also what keeps this experiment’s model to about 1.14 million parameters despite the $32 \times 32 \times 3$ input.

11. Conclusion

A Convolutional Neural Network was implemented in TensorFlow/Keras and trained on the CIFAR-10 dataset. The three-block Conv→Pool architecture ($32 \rightarrow 64 \rightarrow 128$ filters) with a Dense(128)+Dropout(0.5) classification head achieved 73.45% test accuracy and a macro F1-score of 0.74 after 10 training epochs, with 1,143,242 trainable parameters. Vehicle classes (Automobile, Ship, Truck) were classified substantially more reliably than small animal classes (Cat, Dog, Bird), which were frequently confused with one another. Training and validation curves showed mild overfitting emerging after roughly epoch 7, suggesting that additional regularization or early stopping could improve generalization in future runs. Feature map visualizations confirmed the expected CNN behaviour of learning progressively more abstract representations with network depth, from simple edges in the first convolutional layer to highly localized, specific patterns in the third.

12. References

1. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, 1998.
2. A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, University of Toronto, 2009. (CIFAR-10 Dataset)
3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
4. C. Bishop, *Pattern Recognition and Machine Learning*.
5. S. Haykin, *Neural Networks and Learning Machines*.
6. TensorFlow/Keras Documentation.
7. CIFAR-10 Dataset Documentation.